CLAUDE · INSTRUCCIONES · ARCHIVOS · MEMORIA

CONTEXTO organiza tus instrucciones para trabajar mejor con Claude

Cinco cambios con ejemplos de antes y después, un mapa de componentes, un checklist, plantillas, un prompt de auditoría en modo lectura y un método para comparar resultados.

Fecha de consulta de las fuentes: 19 de octubre de 2026.

Qué vas a llevarte

Esta guía te ayuda a ordenar lo que le das a Claude antes de pedirle algo: las instrucciones, los archivos, las herramientas, la memoria y los ejemplos del resultado que quieres. Incluye:

- Una explicación de los cinco cambios que propone el artículo oficial, con un antes y un después de cada uno.
- Un mapa de los componentes del contexto.
- Un checklist para detectar instrucciones duplicadas, contradictorias, obsoletas o innecesarias.
- Una plantilla de `CLAUDE.md`, un ejemplo de skill con referencias separadas y un árbol de carpetas.
- Un prompt de auditoría que trabaja en modo lectura.
- Un método para comparar resultados antes y después sin inventar pruebas.

Los ejemplos usan un proyecto ficticio: **Café Brisa Norte**, una cafetería que quiere una web de pedidos. Todos los datos del proyecto son inventados.

Qué NO es esta guía: no es el artículo (te enlazamos el original y no lo reproducimos), no es una receta de limpieza automática y no ejecuta nada por ti. Nada de lo que aquí se muestra se instaló ni se probó en archivos reales.

El artículo y lo que se pudo verificar

- **Artículo:** The new rules of context engineering for Claude 5 generation models. Fecha de publicación indicada en la página: 24 de julio de 2026. Es una lectura de unos siete minutos, en inglés.
- **Redirección:** al abrir ese enlace, el sitio redirige a una dirección del dominio `claude.dev` donde se muestra el texto. Esta guía no pudo confirmar de forma independiente quién administra ese dominio; sí comprobó que la página enlaza a páginas de `claude.com`, `anthropic.com` y `code.claude.com`, y que el enlace de `claude.com` aparece como publicación oficial en los resultados de búsqueda. Ante la duda, lee también la documentación oficial de Claude Code que se enlaza abajo.
- **Para qué modelos habla:** el artículo se refiere a modelos de la generación Claude 5; menciona por nombre a Claude Opus 5 y Claude Fable 5.
- **Qué dice en una frase:** el contexto (la instrucción del sistema, los archivos de instrucciones, las skills, la memoria y las referencias) pesa más en el resultado que tu mensaje, y varias prácticas antiguas ahora sobran porque los modelos nuevos necesitan menos órdenes y mejores herramientas y referencias.
- **Documentación oficial usada para verificar:** Cómo recuerda Claude tu proyecto (CLAUDE.md y memoria), Skills, Comandos y Buenas prácticas.

Qué significa «reducción del 80 %» y qué no

Según el artículo, el equipo de Anthropic eliminó **más del 80 % de la instrucción del sistema de Claude Code** para modelos como Claude Opus 5 y Claude Fable 5, **sin pérdida medible en sus evaluaciones internas de programación**.

Lo que esto significa:

- Habla de **la instrucción del sistema de un producto concreto** (Claude Code), escrita por quienes lo construyen. No habla de tu `CLAUDE.md`, de tus skills ni de tus prompts.

- Es una afirmación de la propia empresa sobre sus pruebas internas. El artículo no define con qué se mide el tamaño (caracteres, tokens u otra cosa) ni describe las evaluaciones con detalle. Esta guía no pudo replicarla.
- Habla de **qué se quitó y por qué**: órdenes absolutas que ya no hacían falta, ejemplos que limitaban la exploración y texto repetido.

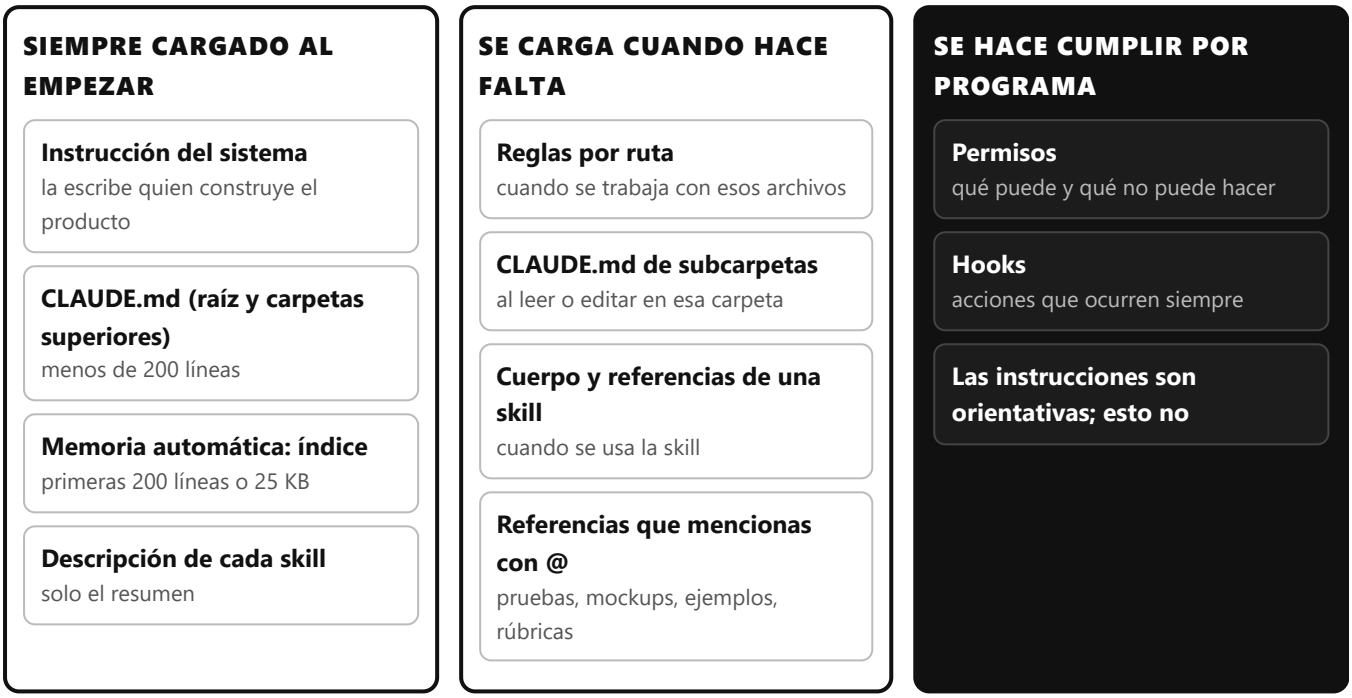
Por qué no implica borrar el 80 % de tus instrucciones:

1. Tus instrucciones son otra capa, con otro contenido: tus convenciones, tus trampas y tus restricciones. La documentación oficial recomienda que cada `CLAUDE.md` tenga menos de 200 líneas y que solo conserve lo que Claude no puede deducir leyendo el proyecto.
2. Un recorte porcentual no distingue entre una instrucción obsoleta y una restricción de seguridad. Quitar la segunda puede costarte un incidente.
3. La regla práctica que la propia documentación propone es una pregunta, no un porcentaje: *«¿Quitar esta línea haría que Claude cometa errores?»* Si no, se puede recortar. Si sí, se queda.

Lo que sí puedes hacer: auditar tus instrucciones con las preguntas de esta guía, proponer cambios, aplicarlos de uno en uno y comparar resultados con tareas reales (apartado «Cómo comparar antes y después»). Si tu archivo termina más corto, es una consecuencia, no el objetivo.

Mapa de los componentes del contexto

Todo lo que Claude «ve» antes de responder viene de estas capas. Cuando algo sale mal, primero pregunta en cuál capa está el problema.



Importar con @ no ahorra contexto: lo importado también se carga al empezar.

CAPA	QUÉ ES	CUÁNDO SE CARGA	QUIÉN LA ESCRIBE
Instrucción del sistema	Las reglas base del producto	Siempre	Quien construye el producto
`CLAUDE.md`	Tus instrucciones de proyecto, personales o de organización	Al empezar la sesión (los de subcarpetas, cuando Claude trabaja con archivos de esa carpeta)	Tú
Reglas por ruta (`.claude/rules/`)	Instrucciones que solo aplican a ciertos archivos	Cuando Claude trabaja con archivos que coinciden con la ruta definida	Tú
Skills	Guías para tareas concretas, con archivos de referencia	Solo la descripción al empezar; el cuerpo, cuando se usa la skill	Tú o tu equipo
Herramientas y conexiones	Acciones que Claude puede ejecutar (nombres, parámetros, descripciones)	Su definición y descripción	Quien las crea
Memoria automática	Notas que Claude guarda sobre tu trabajo	Al empezar la sesión (las primeras 200 líneas o 25 KB del índice)	Claude
Referencias	Archivos que le señalas: pruebas, mockups, código de ejemplo, rúbricas	Cuando los mencionas o los lee	Tú
Permisos y hooks	Reglas que se hacen cumplir por programa	Siempre que ocurre el evento	Tú

Una diferencia importante: según la documentación, Claude trata los archivos `CLAUDE.md` como contexto, **no como configuración que se hace cumplir**. Si una restricción debe cumplirse siempre (por ejemplo, no escribir en una carpeta), la forma fiable es un permiso o un hook, no una frase en el archivo.

Los cinco cambios

Cambio 1. Menos órdenes, más autonomía

Idea sencilla. Antes se escribían prohibiciones absolutas para evitar el peor caso. Los modelos nuevos pueden aplicar un estándar si se lo describes, así que conviene sustituir la lista de «nunca» por la forma de trabajar que esperas. El artículo da un ejemplo: una orden absoluta sobre comentarios en el código pasó a pedir que el código siga el estilo del código que lo rodea.

Antes

```
NUNCA escribas comentarios en el código.
NUNCA hagas funciones de más de 20 líneas.
SIEMPRE usa comillas dobles.
NUNCA toques el archivo de precios.
```

Después

```
Escribe código que siga el estilo del código que lo rodea.
Comenta solo lo que no se entiende leyendo el código.
No modifiques el archivo de precios (src/precios.ts) sin pedirme confirmación.
```

Acción práctica. Haz una lista de tus reglas y clasifícalas en dos grupos: (a) **restricciones** que protegen dinero, datos o permisos (se mantienen, con su razón) y (b) **preferencias de estilo** (se convierten en un estándar). Reescribe solo el grupo (b).

Error habitual. Aflojar también las restricciones de seguridad. Una prohibición sobre credenciales, datos de clientes o borrado de archivos no se «delega» en la autonomía: se conserva y, si debe cumplirse siempre, se refuerza con un permiso o un hook.

Cambio 2. Herramientas e interfaces útiles

Idea sencilla. Es más eficaz diseñar bien la herramienta que explicarla con muchos ejemplos. Nombres claros, parámetros con opciones cerradas y una descripción breve enseñan más que un párrafo de ejemplos. El artículo cuenta que la descripción de una herramienta de tareas pasó de unos 9.100 caracteres a una descripción corta, una lista cerrada de estados y una regla de comportamiento.

Antes (en tu instrucción)

```
Para registrar un pedido usa el script registrar.py. Ejemplo 1: python registrar.py "mesa 4" 2 "americano"
...
Ejemplo 2: ... (diez ejemplos más) ... Si el estado es raro, usa "pendiente".
```

Después

```
Registrar pedidos: usa `python registrar.py --ayuda`. Estados válidos: pendiente | en_preparacion | entrega
do.
```

(y el propio script valida los estados y explica sus errores)

Acción práctica. Para cada herramienta o script que uses: (1) ponle un nombre que diga lo que hace, (2) limita los valores posibles (opciones cerradas), (3) haz que los errores expliquen cómo corregirlos y (4) mueve las instrucciones de uso a la descripción de la propia herramienta.

Error habitual. Repetir en tu `CLAUDE.md` lo que la herramienta ya debería decir por sí misma: duplicas texto y, cuando la herramienta cambia, tus instrucciones quedan obsoletas.

Cambio 3. Contexto dividido en archivos que se consultan cuando hacen falta

Idea sencilla. No metas todo en un archivo que se carga siempre. Deja en `CLAUDE.md` lo que aplica a todo y mueve los procedimientos largos a skills y a archivos de referencia que se cargan cuando se necesitan. Esto se llama *divulgación progresiva*: el detalle aparece cuando hace falta.

Lo que dice la documentación oficial y conviene saber:

- Cada `CLAUDE.md` debería tener **menos de 200 líneas**; los archivos más largos consumen más contexto y reducen que se sigan las instrucciones.
- Las **importaciones** (`@ruta/archivo`) ayudan a organizar, pero **no reducen el costo de contexto**: los archivos importados también se cargan al empezar.
- Las **reglas por ruta** (`.claude/rules/` con `paths:`) y los `CLAUDE.md` de subcarpetas **sí** se cargan solo cuando Claude trabaja con archivos que coinciden.
- El cuerpo de una **skill** se carga solo cuando se usa; al empezar solo se carga su descripción. La documentación recomienda que `SKILL.md` tenga menos de 500 líneas y que el detalle vaya en archivos aparte.

Antes

```
CLAUDE.md (900 líneas): estilo, despliegue paso a paso, guía de reseñas,
plantillas de correo, lista de proveedores, historial de decisiones...
```

Después

```
CLAUDE.md (60 líneas): qué es el proyecto, comandos, convenciones propias, trampas.
.claude/skills/revisar-entrega/SKILL.md: el procedimiento de revisión (se carga al usarlo).
.claude/rules/pedidos.md (con paths): reglas que solo aplican a src/pedidos/.
docs/: material de consulta (se lee cuando se menciona).
```

Acción práctica. Revisa cada bloque de tu `CLAUDE.md` y pregúntate: ¿aplica a todas las tareas? Si no, muévelo: a una skill (un procedimiento), a una regla por ruta (solo para ciertos archivos) o a un documento al que apuntas.

Error habitual. Dividir en archivos pero importarlos todos con `@` desde `CLAUDE.md`: el contenido se carga igual al empezar y no ahorras contexto.

Cambio 4. Memoria: funcionamiento, configuración y límites

Idea sencilla. Claude puede guardar notas por su cuenta (memoria automática), así que ya no hace falta que tú copies cada corrección a mano en `CLAUDE.md`. El artículo lo presenta como un cambio respecto a pedirle que guarde notas manualmente. En `CLAUDE.md` deja lo que sabes antes de empezar una sesión; la memoria recoge lo que se aprende trabajando.

Cómo funciona, según la documentación oficial vigente:

- **Qué guarda:** cuatro tipos de notas: sobre ti (rol y preferencias), correcciones que le das y enfoques que confirmas, decisiones y trabajo en curso que no se pueden deducir del código, y referencias a información fuera del proyecto (por ejemplo, un panel o un gestor de tareas). No guarda lo que puede deducir del código ni lo que ya dice tu `CLAUDE.md`, y no guarda algo en cada sesión.
- **Dónde vive:** en archivos de texto (markdown) dentro de una carpeta por proyecto bajo `~/.claude/projects/`, con un archivo índice `MEMORY.md` y un archivo por nota. Puedes editarlos o borrarlos.
- **Cómo verla y activarla o desactivarla:** el comando `/memory` lista los archivos de instrucciones y de memoria y tiene un interruptor de memoria automática. La documentación indica que está activada por defecto en sesiones locales; se puede desactivar con el ajuste `autoMemoryEnabled` o con la variable de entorno `CLAUDE_CODE_DISABLE_AUTO_MEMORY=1`.
- **Límites:** solo se cargan al empezar las **primeras 200 líneas o 25 KB** de `MEMORY.md` (lo que ocurra primero). La memoria es **local a tu equipo**: no se comparte entre máquinas ni entornos en la nube. La memoria de la conversación principal no se carga en los subagentes.

Antes

```
CLAUDE.md: «Recuerda que el cliente prefiere tono cercano. Recuerda que la semana
pasada corregimos el formato de fechas. Recuerda que el proveedor cambió...» (30 notas sueltas)
```

Después

```
CLAUDE.md: lo estable y verificable (comandos, convenciones, trampas).
Memoria automática: preferencias y correcciones que Claude aprende (revisadas con /memory).
```

Acción práctica. (1) Abre `/memory` y mira qué se guardó. (2) Borra lo incorrecto u obsoleto. (3) Mueve a `CLAUDE.md` lo que debe ser norma del equipo. (4) Si trabajas con datos sensibles, decide si activas o no la memoria para ese proyecto.

Error habitual. Suponer que la memoria es privada o segura por defecto, o que sustituye a las normas del equipo. Es texto en tu disco: no pongas en conversaciones lo que no quieras que quede guardado, y recuerda que no es compartida: lo que el equipo debe saber va en `CLAUDE.md`, en control de versiones.

Cambio 5. Referencias concretas del resultado deseado

Idea sencilla. Una descripción en palabras de lo que quieres es la referencia más débil. Un mockup, una prueba que debe pasar, un código parecido o una rúbrica son referencias que Claude puede abrir y con las que puede comprobar su trabajo. Según el artículo, entre las referencias se prefieren las que son código; un mockup en HTML suele funcionar mejor que una descripción o una captura. La documentación oficial refuerza el mismo punto: dar a Claude una forma de verificar su trabajo (pruebas, una captura para comparar) es lo que más cambia el resultado.

Antes

```
Hazme una página de pedidos moderna y bonita, con buena experiencia de usuario.
```

Después

```
Crea la página de pedidos de Café Brisa Norte.
Referencia visual: @mockups/pedidos.html (sigue su estructura y espaciado).
Debe pasar: @pruebas/pedido-completo.test.js
Revisión: usa @docs/rubrica-pagina.md y dime la puntuación de cada requisito.
```

Acción práctica. Antes de pedir algo, crea o localiza una referencia: una prueba de aceptación, un mockup en HTML, un ejemplo de resultado bueno y una rúbrica corta con requisitos que se puedan comprobar.

Error habitual. Pegar capturas o descripciones largas y no dar una forma de comprobar el resultado. Si no hay nada que ejecutar o comparar, tú te conviertes en la única verificación.

Qué comprobar con /doctor (según versión)

El artículo y la documentación de ayuda no describen `/doctor` con el mismo alcance. Esta tabla separa lo que **dice la documentación oficial de Claude Code consultada el 19 de octubre de 2026** de lo que solo afirma el artículo. No se ejecutó ningún comando en esta guía.

COMANDO	QUÉ HACE SEGÚN LA DOCUMENTACIÓN	VERSIÓN	¿CAMBIA TUS ARCHIVOS?
`/doctor`	Chequeo de la configuración: salud de la instalación, extensiones sin usar o lentas, archivos `CLAUDE.md` (duplicados, contenido que se puede deducir del proyecto, guía que se puede migrar a skills) y permisos.	La documentación no fija una versión mínima para el chequeo general	Informa primero y pide confirmación antes de cambiar nada
`claude doctor` (en la terminal normal)	Diagnóstico de la instalación, de solo lectura, sin iniciar una sesión.	No indicada	No
`/doctor prompt-audit [ruta]`	Audita tus instrucciones: `CLAUDE.md`, `CLAUDE.local.md`, `AGENTS.md` y reglas, skills, comandos, subagentes y estilos de salida bajo `.claude/`. Busca instrucciones pensadas para modelos antiguos, referencias a archivos o comandos que no existen y archivos que se contradicen. Da un informe con ediciones propuestas.	Requiere Claude Code v2.1.283 o posterior	Nada cambia hasta que pidas aplicar los cambios
`/skill-doctor`	Muestra el costo en contexto de tus skills y cuánto se usan, para encontrar las que conviene desactivar.	v2.1.252 o posterior y que la herramienta pueda obtener los indicadores de funciones (feature flags)	Es informativo

Diferencias entre las fuentes que debes tener presentes:

- **Alcance.** El artículo habla de un comando para «ajustar» skills y `CLAUDE.md` y lo describe como una forma de simplificarlos automáticamente. La documentación describe un proceso con **informe y confirmación**: propone y tú autorizas. No promete una limpieza automática universal.
- **Nombre.** El artículo se refiere al comando tanto como `claude doctor` como `/doctor`. En la documentación son cosas distintas: `/doctor` es un chequeo dentro de la sesión y `claude doctor` es un diagnóstico de solo lectura en la terminal.
- **Dónde aplica.** Lo descrito es para Claude Code. Esta guía no encontró documentación que lo extienda a otros productos de Claude.
- **Qué no está verificado.** Que el chequeo funcione igual en tu versión, que encuentre todo lo que te sobra o que sus propuestas sean correctas para tu proyecto. Revisa siempre cada propuesta antes de aceptarla.

Cómo usarlo con prudencia: (1) mira tu versión con `claude --version`, (2) haz una copia o un commit de tus instrucciones antes, (3) ejecuta primero el modo de solo lectura, (4) acepta las propuestas de una en una y (5) comprueba con una tarea real.

Checklist: instrucciones duplicadas, contradictorias, obsoletas o innecesarias

El archivo `checklist-contexto.md` contiene este mismo checklist para copiar. Marca cada punto con una prueba concreta (una línea, un archivo, un comando), no con una impresión.

Duplicadas

- [ ] ¿La misma regla aparece en `CLAUDE.md`, en una skill y en una regla por ruta?
- [ ] ¿Hay un `CLAUDE.local.md` que repite lo que ya dice el del proyecto?
- [ ] ¿Repites en el texto lo que la herramienta ya explica en su descripción?
- [ ] ¿La memoria automática guardó algo que ya está en `CLAUDE.md`?

Contradictorias

- [ ] ¿Hay dos instrucciones que piden cosas incompatibles (por ejemplo, «siempre pregunta» y «nunca interrumpas»)?
- [ ] ¿Un `CLAUDE.md` de subcarpeta contradice al de la raíz?
- [ ] ¿Una skill dice una cosa y una regla por ruta otra?

Obsoletas

- [ ] ¿Hay instrucciones escritas para un modelo anterior (por ejemplo, órdenes absolutas que compensaban un defecto antiguo)?
- [ ] ¿Se mencionan archivos, comandos o carpetas que ya no existen? (compruébalo buscándolos)
- [ ] ¿Hay fechas, versiones o nombres de personas que cambiaron?

Innecesarias

- [ ] ¿Describes lo que Claude puede deducir leyendo el proyecto (estructura de carpetas, dependencias)?
- [ ] ¿Hay frases genéricas como «escribe código limpio»?
- [ ] ¿Hay explicaciones largas o tutoriales que deberían estar en un documento enlazado o en una skill?
- [ ] Para cada línea: ¿quitarla haría que Claude cometa errores? Si no, es candidata a salir.

Que se conservan

- [ ] Restricciones de seguridad, privacidad y permisos (se mantienen, con su razón).
- [ ] Datos que no se pueden deducir del código: trampas, decisiones y requisitos del negocio.
- [ ] Reglas que deben cumplirse siempre: pásalas a un permiso o a un hook en lugar de dejarlas solo como frase.

Plantilla completa de CLAUDE.md (proyecto ficticio)

Plantilla para **Café Brisa Norte** (proyecto ficticio). Copia la estructura, no los datos. Está pensada para quedar muy por debajo de las 200 líneas.

```
# Café Brisa Norte: web de pedidos

## Qué es este proyecto
Web para que los clientes de la cafetería pidan para recoger. Sirve a un solo local.
Los datos de ejemplo son ficticios; no uses datos reales de clientes.

## Comandos
- Instalar: `npm install`
- Servidor local: `npm run dev` (puerto 3000)
- Pruebas: `npm test` (ejecuta antes de dar una tarea por terminada)
- Revisión de estilo: `npm run lint`

## Forma de trabajar
- Escribe código que siga el estilo del código que lo rodea.
- Antes de cambiar algo en varios archivos, resume el plan y espera mi confirmación.
- Para tareas de más de un archivo, empieza por leer, luego propón, luego implementa.
- Al terminar, muestra la salida de las pruebas, no solo «está listo».

## Trampas del proyecto (no se deducen del código)
- Los precios viven en `src/precios.ts` y se actualizan solo con aprobación del dueño.
- El horario de recogida usa la zona horaria del local, no la del servidor.
- El formulario de pedido no debe pedir más datos de los necesarios (nombre y teléfono).

## Seguridad y privacidad (se conservan siempre)
- Nunca escribas claves, tokens ni contraseñas en archivos ni en mensajes. Se leen de variables de entorno.
- No leas ni modifiques `.env`, `secretos/` ni archivos con datos de clientes.
- No ejecutes comandos de borrado masivo ni `git push` sin que yo lo pida.
- No envíes correos, mensajes ni peticiones a servicios externos sin confirmación.

## Dónde buscar el detalle (se consulta cuando hace falta)
- Procedimiento de revisión de entregas: skill `revisar-entrega`.
- Reglas de la carpeta de pedidos: `.claude/rules/pedidos.md`.
- Mockups y ejemplos de resultado: `referencias/`.
- Decisiones del proyecto: `docs/decisiones.md`.
```

Qué se dejó **fuera** a propósito: la lista de carpetas del proyecto, las dependencias y la explicación de cómo funciona el código (Claude puede leerlo), y el detalle de los procedimientos (van en skills). Qué **no** se puede quitar: el bloque de seguridad y privacidad.

Nota: las restricciones de la sección «Seguridad y privacidad» son instrucciones. Para que se cumplan siempre, configura además los permisos de tu entorno (por ejemplo, denegar lecturas de `.env`) o un hook; la documentación recuerda que `CLAUDE.md` es contexto, no configuración que se hace cumplir.

Ejemplo de skill con referencias separadas

Una skill es una carpeta con un archivo `SKILL.md` y, si hace falta, archivos de apoyo. La documentación recomienda poner solo `description` en el encabezado y mantener `SKILL.md` breve. Ejemplo (ficticio):

```
.claude/skills/revisar-entrega/
├── SKILL.md                      # resumen y navegación (corto)
├── referencias/
│   ├── lista-de-seguridad.md     # se lee solo cuando se revisa seguridad
│   └── formato-del-informe.md    # se lee al redactar el informe
└── scripts/
    └── contar-pendientes.py      # se ejecuta, no se carga como texto
```

Contenido de `SKILL.md`:

```
---
name: revisar-entrega
description: Revisa una entrega de la web de pedidos antes de darla por terminada. Úsala cuando pida revisa
r, validar o cerrar una tarea.
---

# Revisar una entrega

1. Ejecuta `npm test` y `npm run lint` y anota el resultado.
2. Compara el resultado con el mockup y la rúbrica que se mencionen en la tarea.
3. Si la tarea toca pedidos, datos de clientes o pagos, lee `referencias/lista-de-seguridad.md`.
4. Redacta el informe con el formato de `referencias/formato-del-informe.md`.
5. No apliques correcciones: lista los hallazgos y espera confirmación.

Si algún paso no se puede ejecutar, dilo con claridad en lugar de suponer que salió bien.
```

Por qué funciona: al empezar solo se carga la descripción; el cuerpo y las referencias se leen cuando la tarea lo pide. Para un flujo con efectos secundarios (por ejemplo, desplegar) la documentación permite impedir que Claude la ejecute por su cuenta con `disable-model-invocation: true`.

Árbol de carpetas para organizar la información

```
cafe-brisa-norte/
├── CLAUDE.md                    # corto: qué es, comandos, forma de trabajar, trampas, seguridad
├── CLAUDE.local.md              # preferencias personales (fuera de control de versiones)
├── .claude/
│   ├── settings.json            # permisos y hooks: lo que DEBE cumplirse siempre
│   ├── rules/
│   │   └── pedidos.md           # reglas solo para src/pedidos/ (con paths)
│   └── skills/
│       └── revisar-entrega/     # procedimiento + referencias + scripts
├── docs/
│   ├── decisiones.md            # por qué se hicieron las cosas (no cabe en CLAUDE.md)
│   └── rubrica-pagina.md        # requisitos comprobables para revisar resultados
├── referencias/
│   ├── pedidos.html             # mockup del resultado deseado
│   └── ejemplo-resultado-bueno.md
├── pruebas/
│   └── pedido-completo.test.js  # prueba de aceptación
└── src/
    └── pedidos/
        └── CLAUDE.md            # opcional: se carga al trabajar en esa carpeta
```

¿Dónde va cada cosa?

¿Debe cumplirse sí o sí?	→ Permiso o hook
¿Aplica a todas las tareas?	→ CLAUDE.md (corto)
¿Solo a ciertos archivos?	→ Regla por ruta o CLAUDE.md de la subcarpeta
¿Es un procedimiento con pasos?	→ Skill (con referencias aparte)
¿Es material de consulta?	→ docs/ o referencias/, y lo señalas con @
¿Es algo que se aprende trabajando?	→ Memoria automática (revísala con /memory)

Prompt de auditoría en modo lectura

El archivo `prompt-auditoria-contexto.md` contiene este prompt. Está escrito para no modificar nada: solo lee, informa y propone. Sustituye los campos entre corchetes.

```
Actúa como auditor de instrucciones. Trabaja en MODO LECTURA: no modifiques, crees, muevas ni borres ningún archivo, no ejecutes comandos que cambien algo y no instales nada. Si necesitas ejecutar algo, dímelo y espera.

Carpeta a auditar: [RUTA DEL PROYECTO]
Archivos de instrucciones: CLAUDE.md, CLAUDE.local.md, AGENTS.md, .claude/rules/, .claude/skills/, .claude/agents/ y los que yo indique: [OTROS ARCHIVOS].
Modelo y versión que uso: [MODELO Y VERSIÓN].

Tarea:
1. Haz un inventario: lista cada archivo con su número de líneas y cuándo se carga (siempre, por ruta o al usarlo).
2. Revisa cada instrucción y clasifícala en una de estas categorías:
   - Duplicada (dónde está repetida).
   - Contradictoria (con qué otra instrucción choca).
   - Obsoleta (menciona archivos, comandos o conductas que ya no existen o fueron pensadas para modelos anteriores; verifica leyendo, no supongas).
   - Innecesaria (puede deducirse leyendo el proyecto, es genérica o está explicada en una herramienta).
   - Necesaria (la quita y el trabajo falla) o De seguridad/privacidad/permisos (se conserva siempre).
3. Para cada hallazgo entrega: archivo y línea, cita corta, categoría, evidencia (qué leíste para afirmarlo) y propuesta: conservar, reescribir como estándar de trabajo, mover a skill o regla por ruta, convertir en permiso o hook, o eliminar.
4. No propongas quitar ninguna restricción de seguridad, privacidad o permisos. Si una instrucción importante solo existe como frase, propón reforzarla con un permiso o un hook.
5. Entrega un informe con: resumen (conteos por categoría), tabla de hallazgos ordenada por impacto, propuesta de nueva estructura de carpetas y 3 cambios para empezar.
6. Si no puedes comprobar algo, escribe «no verificado» y explica qué te falta. No inventes archivos, líneas ni resultados.
7. No apliques ningún cambio. Termina preguntándome qué cambios aprobar, de uno en uno.
```

Cómo comparar antes y después

Para saber si ordenar tus instrucciones mejoró algo, compara con tareas reales y registra lo que ocurre. No se trata de demostrar nada: se trata de observar. **No inventes resultados ni los presentes como testimonios**: solo vale lo que tú midas.

1. **Congela el punto de partida.** Guarda una copia (o un commit) de tus instrucciones actuales y anota la versión de Claude Code y el modelo.
2. **Elige entre 5 y 8 tareas reales** y repetibles (por ejemplo, «añadir un campo al formulario de pedido»). Escríbelas tal cual y no las cambies entre pruebas.
3. **Define los requisitos de aceptación** de cada tarea antes de empezar: una lista de comprobaciones que se cumplen o no (pruebas que pasan, archivo en el lugar correcto, sin tocar los archivos prohibidos).
4. **Ejecuta cada tarea al menos dos o tres veces** con las instrucciones actuales y registra los resultados. Los resultados de modelos varían entre ejecuciones.
5. **Aplica un solo cambio** (por ejemplo, mover un procedimiento a una skill) y repite las mismas tareas en sesiones nuevas.
6. **Anota los mismos datos** en cada ejecución:

TAREA	VERSIÓN DE INSTRUCCIONES	REQUISITOS CUMPLIDOS (DE N)	CORRECCIONES QUE HICISTE	TAMAÑO DEL CONTEXTO AL EMPEZAR	OBSERVACIONES
Añadir campo «notas»	Antes				
Añadir campo «notas»	Después				

7. **Mide el contexto, no lo supongas.** La documentación oficial indica que `/context` muestra lo que se cargó (incluidos los archivos de memoria) y que `/doctor` estima el costo de la lista de skills. Registra lo que muestren en tu versión.
8. **Si es posible, compara a ciegas:** pídele a otra persona que revise los resultados sin saber cuáles son «antes» y «después».
9. **Conclusión honesta.** Si hubo mejora en una tarea y no en otra, dilo así. Si no hubo diferencia, también es un resultado válido: tus instrucciones quedaron más cortas sin empeorar. Si empeoró algo, revierte ese cambio.

Lo que se conserva siempre: seguridad, privacidad y permisos

Ordenar el contexto no es aflojar los controles. Antes de recortar:

- Conserva las prohibiciones sobre credenciales, datos personales, borrado de archivos y envío de mensajes.
- Refuerza lo crítico con permisos y hooks. Según la documentación, los hooks son deterministas, mientras que las instrucciones son orientativas.
- Revisa qué guarda la memoria y decide si debe estar activada en proyectos con datos sensibles.
- Prueba cualquier cambio en una copia o una rama, y solo después aplícalo al proyecto real.
- No ejecutes automáticamente las propuestas de una herramienta: léelas.

Primeros pasos

1. Copia tus instrucciones actuales a una carpeta de respaldo.
2. Ejecuta el prompt de auditoría en modo lectura.
3. Aplica un cambio (por ejemplo, convertir cinco prohibiciones de estilo en un estándar) y compara con una tarea real.
4. Mueve un procedimiento largo a una skill.
5. Revisa `/memory` y limpia lo que sobre.
6. Si tu versión lo permite, prueba `/doctor prompt-audit` como segunda opinión y revisa cada propuesta antes de aceptarla.

Fuentes y verificación

Fecha de consulta: 19 de octubre de 2026.

- Artículo: The new rules of context engineering for Claude 5 generation models (fecha indicada en la página: 24 de julio de 2026).
- Documentación de Claude Code: Memoria y CLAUDE.md, Skills, Comandos, Buenas prácticas.
- Qué se hizo: se leyeron el artículo y las cuatro páginas de documentación el 19 de octubre de 2026. No se ejecutó ningún comando, no se instaló nada, no se modificaron archivos reales y no se midió el efecto de ningún cambio. Las cifras (200 líneas, 25 KB, 500 líneas, versiones mínimas) son las que indica la documentación en esa fecha y pueden cambiar. El texto del artículo no se reproduce: se resumen sus ideas con palabras propias.